YAMLess — Architectural Evolution

YAMLess: Architectural Evolution

How YAMLess should evolve to prevent the gaps found in manual analysis

---

Background

A manual analysis of `krogertechnology/pos-elera-sales-compliance` found key divergences between YAMLess-generated infrastructure and the app's actual requirements. The root cause is structural — fixing gaps piecemeal will keep producing the same class of problems on the next repo.

The Three Failure Modes

Failure Mode 1: Shallow reads of sources the analyzer already touches

The analyzer reads Helm `values*.yaml`, Dockerfile, and Spring Boot YAML — but only for the signals it was originally designed to find (secret names, runtime, port). Those same files contain resource limits, replica counts, health paths, secondary ports, and JVM flags. The data is there; the reader ignores it.

What's needed: a full Helm values extraction pass

Instead of `env_scanner.py` cherry-picking secret keys from Helm, a proper `helm_reader.py` should extract the full relevant surface:

```
resources.limits.memory / resources.requests.memory
replicaCount / autoscaling.minReplicas / autoscaling.maxReplicas
livenessProbe.httpGet.path / readinessProbe.httpGet.path
ports (all of them, not just the first)
javaOpts / env[].value (for -Xmx, -Dcom.sun.jmx.*)
```

This single investment closes #28 (health path), #29 (memory limits), #31 (min replicas), and #32 (JMX port) simultaneously, because they're all in the same file. It also makes YAMLess **respect the existing Helm configuration** rather than overwriting operator knowledge with defaults.

Similarly, the Dockerfile analyzer reads one `EXPOSE` line. It should collect all of them and feed `additional_ports`.

Failure Mode 2: Runtime is known but not used for runtime-specific enrichment

The analyzer detects `runtime: java` early, then proceeds with the same generic logic regardless. Java Spring Boot apps have a well-known ecosystem of signals that only make sense to look for once you know the runtime:

Signal	Where it lives	What it means
`spring-boot-starter-actuator`	pom.xml	health path is `/actuator/health`

Signal	Where it lives	What it means
`spring-cloud-starter-config`	pom.xml	startup dependency on external config server
`management.server.port`	application.yml	secondary port needed
`-Xmx` in `JAVA_OPTS`	Dockerfile / values.yaml	minimum container memory = 1.5× heap
`-Dcom.sun.jmx.remote.port`	Dockerfile / values.yaml	JMX port needed
`spring-boot-starter-amqp` + direct reply-to	source code	Service Bus migration risk

What's needed: a runtime enrichment pass

After the generic dependency scan, run a runtime-specific enrichment step. `java_enricher.py` fires when `runtime == JAVA` and mines the Spring-specific signals above. .NET would have its own enricher for Kestrel ports, health checks via `MapHealthChecks`, and `IConfiguration` patterns.

This is the structural fix for #28, #29, #30, #32, #33 — not individual workarounds but a single extensible enrichment layer.

Failure Mode 3: Technology mapping with no migration risk table

When YAMLess maps `spring-rabbit` → Azure Service Bus, it's correct at the infrastructure provisioning level. But it has no concept of protocol compatibility between source and target. Some mappings are clean; others have known gotchas:

Source	Azure target	Known risks
RabbitMQ / Spring AMQP	Service Bus	Direct reply-to unsupported; AMQP 0-9-1 vs 1.0 differences
MongoDB	Cosmos DB for MongoDB	Aggregation pipeline coverage gaps; no transactions pre-4.0
Redis Pub/Sub	Azure Cache for Redis	Pub/Sub supported but keyspace notifications differ
Spring Cloud Config	—	No Azure equivalent; needs init container or sidecar

What's needed: a migration risk table in the analyzer

A `migration_risks.py` module with a static table keyed on `(source_technology, target_cloud)`. When a risk entry exists for a detected dependency, it gets attached to the profile as a warning — surfaced in the CLI output, the report, and eventually the generated Terraform as a comment. This turns silent failures into explicit migration guidance.

Failure Mode 4: LLM gap-filler is underutilized

The LLM gap-filler currently fires only when confidence is `low` or `medium` on structural fields (runtime, shape, secrets). But the hardest gaps — AMQP direct reply-to, Spring Cloud Config server topology, JVM memory tuning — are **semantic questions about application behavior**, not confidence gaps on profile fields.

What's needed: semantic gap-filling questions

The gap-filler should have a second mode: given a detected technology combination, ask targeted semantic questions by reading source files:

- Java + Spring AMQP detected → "Does this app use `amq.rabbitmq.reply-to` or `RabbitTemplate.convertSendAndReceive()`?"
- Java + `CLOUD_CONFIG_URL` detected → "Is the config server in-cluster or external? What is its service name?"
- Java + large `-Xmx` detected → "What is the full JVM startup command and expected heap at steady state?"

These aren't low-confidence fields — they're questions the static analyzer structurally cannot answer. The LLM is exactly the right tool, but it needs to be prompted for these cases rather than waiting for a confidence signal.

Failure Mode 5: No concept of existing vs. greenfield workload

Every run generates a full cluster — VNet, AKS control plane, node pools, identity federation — even when the app is being migrated into an existing cluster. For Kroger's context, the Terraform is largely noise; only `k8s/` is actionable.

What's needed: a `--workload-only` mode

A flag that skips cluster provisioning modules and renders only: - `k8s/` manifests (Deployment, Service, Ingress, HPA, NetworkPolicy, SecretProviderClass) - Key Vault + secrets (the app's backing secrets, not the cluster's) - A `terraform/` stub that references data sources for the existing cluster rather than creating it

The profile and renderer already separate these concerns cleanly. This is a renderer-level switch, not a structural change.

Target Architecture

The current pipeline is:

```
Static Analyzer → [LLM if low confidence] → Golden Resolver → Renderer → Validator
```

It should become:

```
Static Analyzer
    ↓
Runtime Enricher          (Java / .NET / Node — runtime-specific signal extraction)
    ↓
Helm / Manifest Reader    (resources, replicas, health paths, all ports)
    ↓
Migration Risk Evaluator  (source tech × target cloud → warnings)
    ↓
[LLM: structural gaps + semantic questions]
    ↓
Golden Resolver → Renderer → Validator
```

The first three new stages are all static — no API key, no cost, deterministic. They turn YAMLess from a **best-effort inference tool** into one that **reads what operators already know and respects it**, only falling back to defaults when the information genuinely isn't there.

The manual analysis found issues because YAMLess defaulted when it didn't need to — the Helm chart had the memory limit, replica count, health path, and JMX port. The analyzer just wasn't reading that far.

Relationship to Open Issues

Fixes #28–#33 were filed and three blockers (#28, #29, #31) have already been implemented. The architectural work (#37) provides the structural home for all of them.

Issues	Relationship to #37
#28, #29, #30, #31, #32, #33	Will be **absorbed** — these fixes find their permanent structural home inside the Runtime Enricher, Helm Deep Reader, and Migration Risk layer when #37 lands. The behavior stays; the code moves. Not wasted work — it's incremental delivery.
#34 (secret denylist)	**Independent** — lives in `env_scanner.py` regardless of architecture
#35 (Cosmos DB module)	**Independent** — renderer work, entirely unaffected
#36 (secret count warning)	**Independent** — small addition to `runner.py`

Does #37 need to come before the 9 issues? No. The 9 are targeted patches; #37 is a refactor. Ship the patches now — three of them (#28, #29, #31) are blockers that cause pods to OOM or restart-loop.

Does #37 render any of the 9 useless? No. The practical implication: when implementing #28–#33, write the code knowing it will eventually be refactored into the new structure. Keep each fix contained so the eventual absorption into #37 is clean rather than a rewrite.

YAMLess — https://github.com/zafar-ahmed-kr/YAMLess Date: 2026-11-11